ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

TEKNOLOJİ FAKÜLTESİ

ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ BÖLÜMÜ

MAKİNE ÖĞRENMESİNE GİRİŞ DERSİ

SUNUM 4 RAPORU

MAKİNE ÖĞRENMESİNDE HİPERPARAMETRELER VE MODERN EĞİTİM TEKNİKLERİ

2312705056

ÇAĞLA EYLÜL ÇETİN

Dersin Öğretim Üyesi

Dr. Öğr. Üyesi Ali ŞENTÜRK

1.GİRİŞ

Makine öğrenmesi, verilerden öğrenerek tahmin yapabilen sistemlerin geliştirilmesini sağlayan önemli bir yapay zekâ alanıdır. Bu sistemlerin başarısı yalnızca kullanılan model yapısına değil, aynı zamanda modelin eğitim sürecinde kullanılan hiperparametrelere de bağlıdır. Hiperparametreler, modelin öğrenme sürecini yönlendiren ve eğitim başlamadan önce belirlenen ayarlardır.

Hiperparametrelerin doğru seçilmesi, modelin performansını önemli ölçüde etkileyebilir. Yanlış ayarlamalar modelin düşük doğrulukla çalışmasına veya öğrenme sürecinin dengesiz olmasına neden olabilir. Bu nedenle hiperparametre optimizasyonu, makine öğrenmesi uygulamalarında kritik bir rol oynamaktadır.

Günümüzde hiperparametre tuning, Batch Normalization, Experiment Tracking ve veri hatalarının (data leakage) önlenmesi gibi modern eğitim teknikleri, makine öğrenmesi modellerinin başarısını artırmak için önemli bir rol oynamaktadır. Bu çalışmada, hiperparametreler ve modern eğitim teknikleri ele alınmış, özellikle Batch Normalization yönteminin model performansı üzerindeki etkisi incelenmiştir. Ayrıca yapılan mini proje ile bu yöntemin uygulamadaki sonuçları gösterilmiştir.

2.TEMEL KAVRAMLAR

2.1 HİPERPARAMETRELER

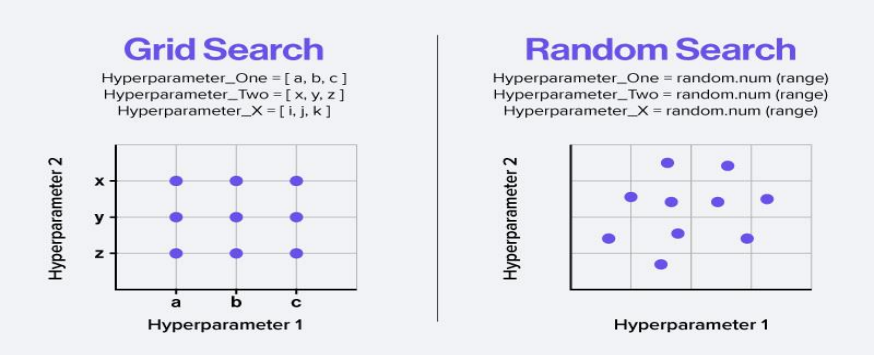
Hiperparametreler, makine öğrenmesi modellerinin eğitim sürecini kontrol eden ve model performansını doğrudan etkileyen temel ayarlardır. Bu parametreler, modelin nasıl öğreneceğini doğrudan etkiler ve modelin kendi içinde öğrenilen parametrelerden farklıdır. Model parametreleri eğitim sırasında otomatik olarak belirlenirken, hiperparametreler önceden seçilir ve kullanıcı tarafından ayarlanır.

Sinir ağlarında katman sayısı, düğüm sayısı ve öğrenme oranı gibi değerler hiperparametrelere örnek olarak verilebilir. Bu tür ayarlar, modelin karmaşıklığını, öğrenme hızını ve genel performansını belirleyen önemli faktörlerdir. Bu nedenle hiperparametrelerin uygun şekilde seçilmesi, modelin hem öğrenme hızını hem de doğruluğunu doğrudan etkilediği için bu parametrelerin optimize edilmesi büyük önem taşımaktadır.

Grid Search ve Random Search

Grid Search yöntemi, belirlenen hiperparametre değerlerinin tüm olası kombinasyonlarını sistematik bir şekilde dener. Bu sayede en iyi sonucu veren kombinasyon bulunabilir. Ancak tüm ihtimallerin denenmesi, özellikle çok sayıda hiperparametre olduğunda zaman açısından maliyetli olabilir.

Random Search ise hiperparametre değerlerini rastgele seçerek denemeler yapar. Tüm kombinasyonları denemek yerine belirli sayıda deneme ile iyi sonuçlar elde etmeye çalışır. Bu yöntem, özellikle geniş hiperparametre aralıklarında daha hızlı ve verimli sonuçlar verebilir.[3]



2.2 Batch Normalization

Batch Normalization, sinir ağlarında eğitim sürecini daha hızlı ve stabil hale getirmek amacıyla kullanılan bir yöntemdir.[2] Bu teknik, her katmana gelen verileri belirli bir ortalama ve varyans değerine göre normalize ederek verilerin daha dengeli bir dağılıma sahip olmasını sağlar. Eğitim sırasında katmanların giriş dağılımlarının sürekli değişmesi "içsel kovaryat kayması" olarak adlandırılır ve bu durum öğrenme sürecini olumsuz etkileyebilir. Batch Normalization, bu değişimi azaltarak modelin daha kararlı öğrenmesini sağlar. Böylece model daha yüksek öğrenme oranlarıyla çalışabilir, eğitim süresi kısalır ve aşırı öğrenme gibi problemlerin etkisi azaltılabilir.

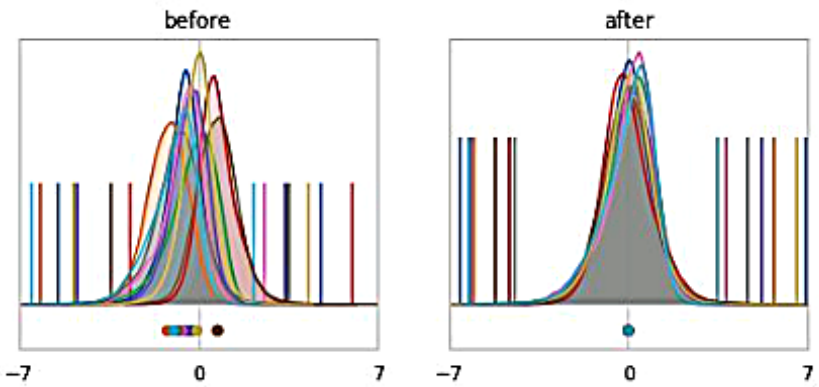
$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

$$y = \gamma \hat{x} + \beta$$

Bu formülde:

- x: giriş verisi
- μ: ortalama
- $\sigma 2$: varyans
- ϵ: sıfıra bölünmeyi önlemek için küçük bir değer
- γ,β: öğrenilebilir parametrelerdir

İlk adımda veriler normalize edilir, ardından ölçeklendirme ve kaydırma işlemi uygulanarak modelin öğrenmesine uygun hale getirilir.



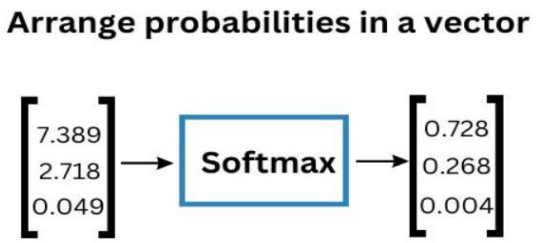
2.3 SOFTMAX VE ÇOK SINIFLI SINIFLANDIRMA

Softmax aktivasyon fonksiyonu, sinir ağlarında özellikle çok sınıflı sınıflandırma problemlerinde kullanılan önemli bir fonksiyondur. Bu fonksiyon, modelin ürettiği ham çıktı değerlerini (logitleri) olasılık dağılımına dönüştürerek her bir sınıfa ait olasılık değerleri elde edilmesini sağlar.[1] Softmax çıktıları 0 ile 1 arasında değer alır ve tüm çıktıların toplamı 1'e eşittir, bu da modelin hangi sınıfa daha yüksek olasılık verdiğini açık bir şekilde göstermesine imkan tanır.

Matematiksel olarak Softmax fonksiyonu, her bir değerin üstelinin alınması ve tüm değerlerin üstellerinin toplamına bölünmesiyle hesaplanır. Bu sayede büyük değerler daha belirgin hale gelir ve modelin karar verme süreci daha net olur. Ayrıca türevlenebilir bir yapıya sahip olduğundan geri yayılım algoritması ile birlikte etkin bir şekilde çalışarak modelin öğrenmesini destekler. Bu formül, model çıktılarının olasılık dağılımına dönüştürülmesini sağlar.

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{N} e^{z_j}}$$

- *zi* : i. sınıfa ait model çıktısı (logit değeri)
- e: Euler sayısı (yaklaşık 2.718)
- *N*: toplam sınıf sayısı
- $\sum_{j=1}^{N} e^{z_j}$: tüm sınıfların üstel değerlerinin toplamı
- *σ(zi)*: i. sınıfa ait olasılık değeri



Softmax fonksiyonu ile modelin ham çıktı değerlerinin olasılık dağılımına dönüştürülmesi.

2.4 Experiment Tracking

Deney takibi, makine öğrenmesi projelerinde model eğitimi sırasında kullanılan hiperparametrelerin, veri setlerinin ve elde edilen performans sonuçlarının sistemli bir şekilde kaydedilmesini sağlayan bir süreçtir. Bu sayede yapılan denemeler karşılaştırılabilir hale gelir ve model geliştirme süreci daha düzenli ve tekrar edilebilir olur.

Bu süreçte kullanılan araçlardan biri MLflow olup deneylerin kaydedilmesi, model sonuçlarının karşılaştırılması ve model versiyonlarının yönetilmesini sağlar. Weights & Biases (W&B) ise deneylerin görselleştirilmesi ve performans metriklerinin izlenmesine olanak tanır. Bu araçlar sayesinde model geliştirme süreci daha kontrollü ve verimli bir şekilde yürütülebilir.[6]

2.5 DATA LEAKAGE

Data leakage, makine öğrenmesi modelinin eğitim süreci sırasında test verisine ait bilgilere dolaylı ya da doğrudan erişmesi durumudur. Bu durum, modelin gerçek performansını olduğundan daha yüksek göstermesine neden olur. Model eğitim verisi dışında bilgiye ulaştığında, yeni ve gerçek veriler üzerinde doğru tahminler yapamaz ve genelleme yeteneği düşer.

Bu problem genellikle veri hazırlama aşamasındaki hatalardan kaynaklanır. Örneğin, modelin geleceğe ait bilgileri içeren verilerle eğitilmesi veya eğitim ve test verilerinin doğru şekilde ayrılmaması data leakage oluşmasına neden olabilir. Böyle durumlarda model, test verisini öğrenmiş gibi davranır ancak gerçek dünyada başarısız sonuçlar üretir.[5]

Data Leakage’a Neden Olan Hatalar

- Eğitim ve test verisinin karıştırılması
- Gelecek bilgilerin modele dahil edilmesi
- Veri ön işleme işlemlerinin tüm veri üzerinde yapılması

3.YÖNTEM / MODEL AÇIKLAMASI

Bu çalışmada, makine öğrenmesi modelinin performansını değerlendirmek amacıyla hiperparametre ayarlama teknikleri ve modern eğitim yöntemleri ile birlikte ele alınmıştır. Model eğitimi öncesinde veri seti normalize edilerek öğrenme sürecine uygun hale getirilmiştir.

Model yapısı oluşturulurken tam bağlı (dense) katmanlardan oluşan bir yapay sinir ağı kullanılmıştır. Modelin öğrenme sürecini daha stabil hale getirmek için Batch Normalization yöntemi uygulanmıştır. Ayrıca, çok sınıflı sınıflandırma problemi için çıkış katmanında Softmax aktivasyon fonksiyonu tercih edilmiştir.

Çalışma kapsamında Batch Normalization kullanılan ve kullanılmayan iki farklı model oluşturulmuş ve aynı veri seti üzerinde eğitilmiştir. Böylece bu yöntemin model performansı üzerindeki etkisi doğrudan karşılaştırılmıştır.

4. UYGULAMA ALANLARI

Makine öğrenmesinde hiperparametre ayarlama ve modern eğitim teknikleri, gerçek hayattaki birçok uygulamada model performansını artırmak için kullanılmaktadır. Örneğin, yüz tanıma sistemlerinde doğru hiperparametre seçimi modelin kişileri daha doğru tanımasını sağlar. Benzer şekilde e-ticaret sitelerinde kullanılan öneri sistemlerinde, model ayarlarının doğru yapılması kullanıcıya daha isabetli öneriler sunulmasına yardımcı olur.

Doğal dil işleme alanında, duygu analizi veya metin sınıflandırma gibi uygulamalarda modelin daha hızlı ve dengeli öğrenmesi için bu tekniklerden yararlanılır. Ayrıca sağlık alanında yapılan çalışmalarda veri hatalarının (data leakage) önlenmesi, modelin güvenilir sonuçlar üretmesi açısından oldukça önemlidir.

5. YAPILAN MİNİ PROJE AÇIKLAMASI

5.1 Kullanılan Veri Seti

Bu çalışmada MNIST veri seti kullanılmıştır. MNIST, 0’dan 9’a kadar el yazısı rakam görüntülerinden oluşan yaygın bir veri setidir. Veri seti 60.000 eğitim ve 10.000 test örneği içermektedir.[4] Görüntüler 28x28 boyutundadır. Görsel veri

içermesi ve sınıflandırma problemleri için yaygın olarak kullanılması nedeniyle bu çalışmada tercih edilmiştir.

5.2 Kullanılan Yöntem: Bu çalışmada Batch Normalization yönteminin model performansı üzerindeki etkisini incelemek amacıyla iki farklı yapay sinir ağı modeli oluşturulmuştur. İlk modelde Batch Normalization kullanılmamış, ikinci modelde ise gizli katmanlara Batch Normalization eklenmiştir.

Her iki model aynı veri seti üzerinde, aynı eğitim parametreleri kullanılarak eğitilmiştir. Böylece Batch Normalization yönteminin modelin öğrenme süreci ve doğruluk performansı üzerindeki etkisi karşılaştırılmıştır. Çıkış katmanında çok sınıflı sınıflandırma için Softmax aktivasyon fonksiyonu kullanılmıştır.

5.3 Kullanılan Kodlamalar : Bu çalışmada Python programlama dili, Jupyter Notebook ortamında kullanılmıştır. Veri ön işleme, model oluşturma ve model eğitimi işlemleri TensorFlow/Keras kütüphaneleri ile gerçekleştirilmiştir. Elde edilen sonuçların görselleştirilmesi için matplotlib kütüphanesinden

Veri Ön İşleme Kodu

```
X_train = X_train / 255.0

X_test = X_test / 255.0
```

Model Oluşturma Kodu

```
model_with_bn = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    BatchNormalization(),
    Dense(64, activation='relu'),
    BatchNormalization(),
    Dense(10, activation='softmax')         ])
```

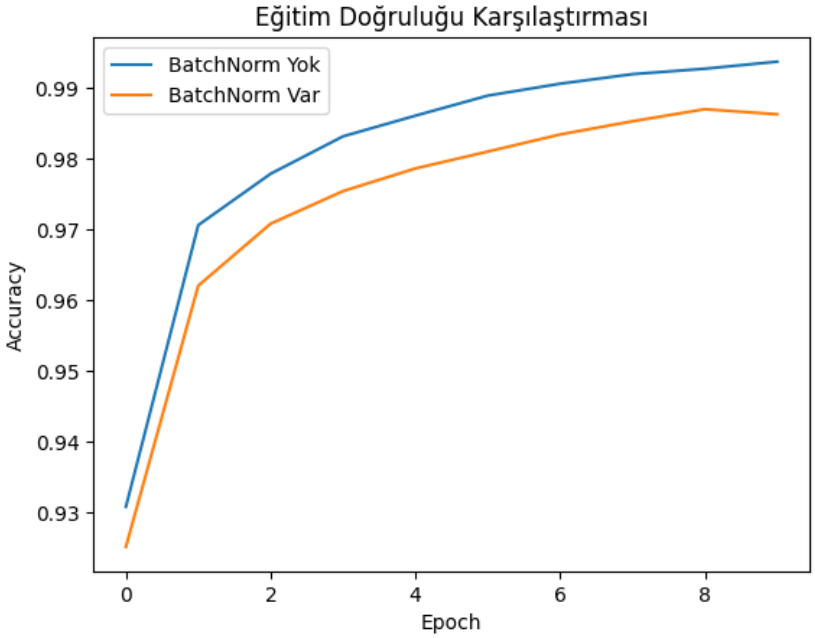
Model Eğitimi Kodu

```
history_with_bn = model_with_bn.fit(
    X_train, y_train,
    epochs=10,
    validation_data=(X_test, y_test),
    verbose=1
 )
```

5.4 Elde Edilen Sonuçlar ve Program Çıktıları

Eğitim Doğruluğu Karşılaştırılması

Model eğitimi sonucunda elde edilen doğruluk değerleri grafik üzerinde gösterilmiştir. Aşağıdaki grafikte Batch Normalization kullanılan ve kullanılmayan modellerin eğitim sürecindeki doğruluk değişimi karşılaştırılmaktadır.



Bu grafiğe bakıldığında her iki modelin de eğitim sürecinde doğruluk değerlerini artırdığı görülmektedir. Batch Normalization kullanılan modelin daha dengeli bir öğrenme eğrisi sergilediği, ancak doğruluk açısından büyük bir fark oluşmadığı gözlemlenmiştir.

Test Sonuçlarının Karşılaştırılması

Eğitim tamamlandıktan sonra modeller test verisi üzerinde değerlendirilmiş ve elde edilen doğruluk sonuçları aşağıda verilmiştir. Sonuçlara göre iki model de yüksek doğruluk elde etmiştir. Batch Normalization kullanılan modelin küçük bir farkla daha iyi performans gösterdiği görülmektedir.

```
BatchNorm YOK Accuracy: 0.9768000245094299

BatchNorm VAR Accuracy: 0.978600025177002
```

6. SONUÇ

6.1 Genel Değerlendirme

Bu çalışmada hiperparametre ayarlama yöntemleri ve modern eğitim tekniklerinin model performansı üzerindeki etkisi incelenmiştir. Yapılan mini projede Batch Normalization kullanılan modelin küçük bir farkla daha iyi doğruluk elde ettiği görülmüştür.

6.2 Öğrenilenler

Bu çalışma sonucunda hiperparametrelerin ve veri ön işleme adımlarının model performansını doğrudan etkilediği anlaşılmıştır. Batch Normalization yönteminin modelin daha stabil öğrenmesini sağladığı görülmüştür.

7. KAYNAKÇA

[1] I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016.

[2] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” *ICML*, 2015.

[3] J. Bergstra and Y. Bengio, “Random Search for Hyper-Parameter Optimization,” *Journal of Machine Learning Research*, 2012.

[4] TensorFlow, “MNIST Dataset,” [Online]. Available: https://www.tensorflow.org

[5] APIUP, “Data Leakage Nedir ve Nasıl Önlenir?”, [Online]. Available: https://apiup.com.tr/veri-bilimi-projelerinde-data-leakage-nedir-ve-nasil-onlenir-stratejik-ipuclariyla-basariya-ulasin

[6] Ultralytics, “Experiment Tracking Nedir?”, [Online]. Available: https://ultralytics.com/tr/glossary/experiment-tracking